OS LAB 04

Name: Muhammad Ghulam Mujtaba Qureshi

Roll No 24K-0535

Question 1 (Screenshots)

Ghulam Mujtaba Qureshi 24K-0535

OS-LAB TASK 04

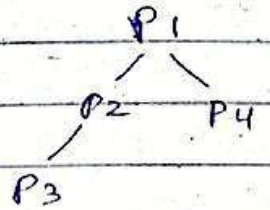
Q1

(1) After first fork() = 2 processes, after second fork total will be 4 processes

(2) As there are 4 processes so 4 times the print function will be executed.

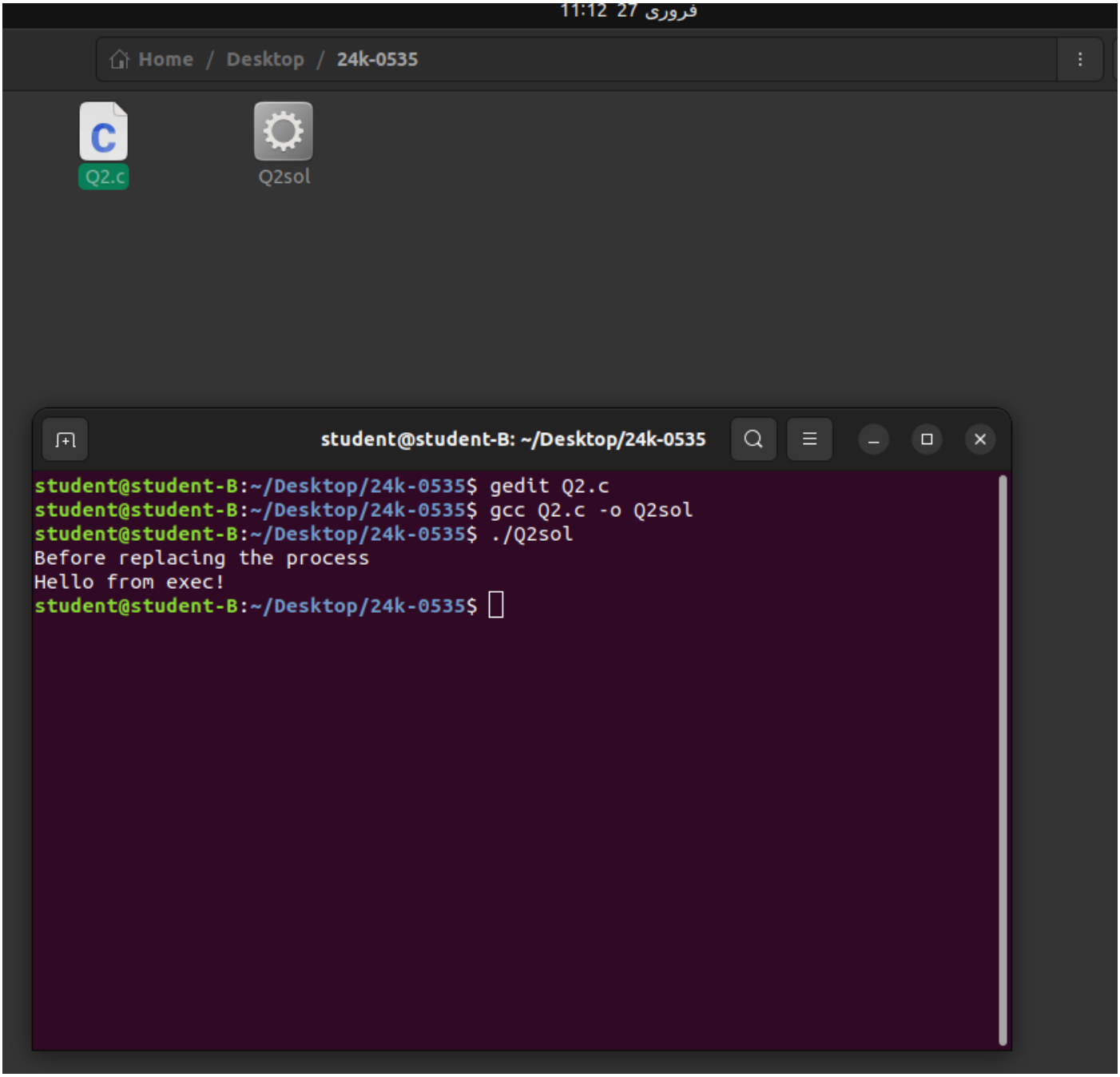
(3) when 2 processes = a=1, → a=a+1 = a=2
then 4 processes so a=a+2 → a=4,
there is no variation, the range of value printed will only be 4.

(4)



(5) No, we cannot predit the output as the cpu decide randomly which process to print first.

Question 2 (Screenshots)



24K-0535

Q2

(1) excel() replaces the current running process with another program, in this code it replaces the program with /bin/echo

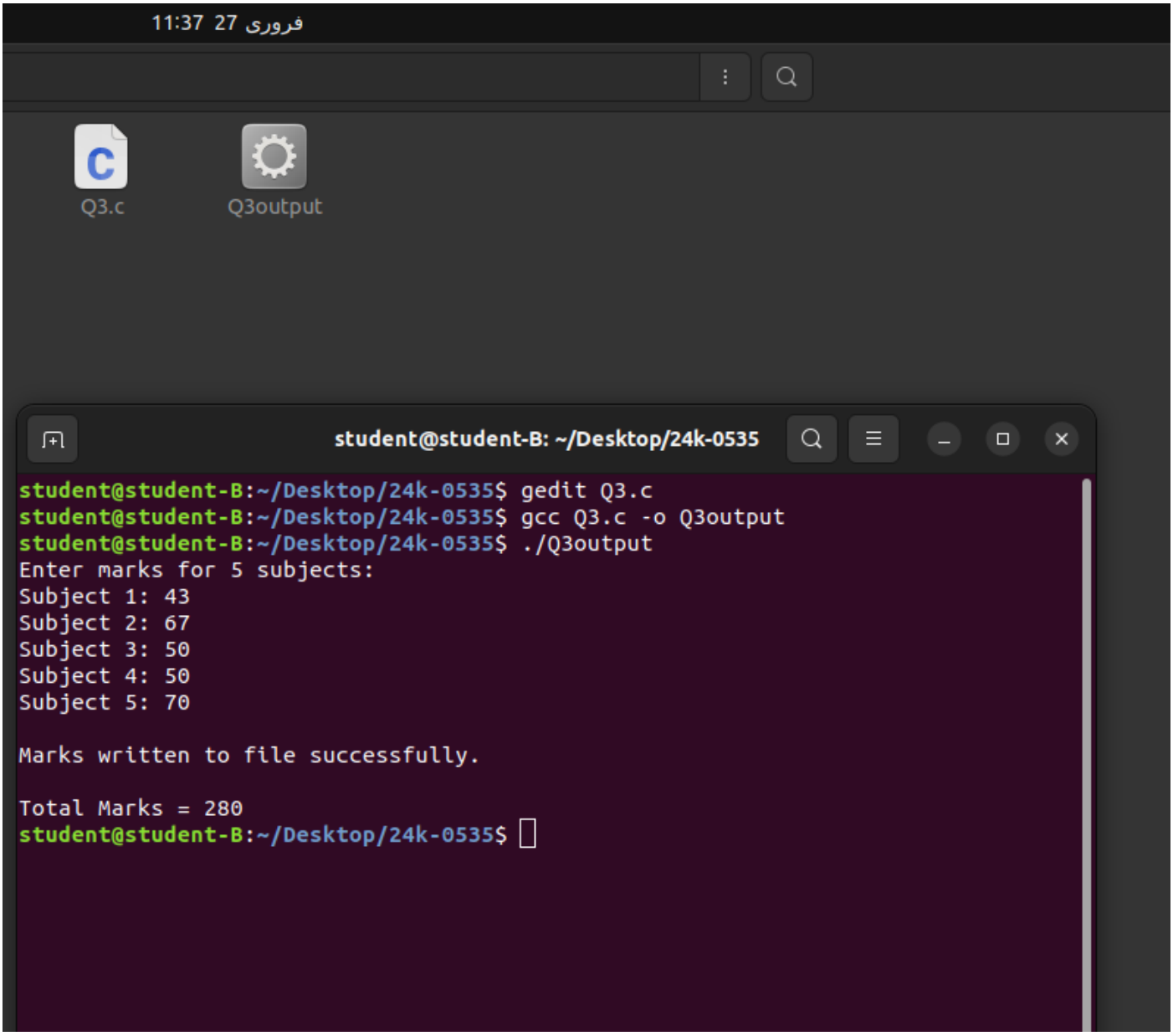
(2) output:
Before replacing the process
Hello from exec!

(3) Because if excel() is successful, it completely replaces the process and never return to the original code

(4) If /bin/echo path is incorrect, execl() fails and return -1, so the second printf() will execute.
Output
"Before replacing the process
After exec - will this line run"

(5) If we remove excel(), so both printf() statements will run normally.

Question 3 (Screenshots)



Question 3 (Code)

Text Editor 11:38 27 فروری

Open Q3.c ~/Desktop/24k-0535

```c
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    int marks[5];
    char buffer[100];
    int total = 0;

    fd = open("marks.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Error creating file");
        return 1;
    }

    printf("Enter marks for 5 subjects:\n");
    for (int i = 0; i < 5; i++) {
        printf("Subject %d: ", i + 1);
        scanf("%d", &marks[i]);

        sprintf(buffer, "%d\n", marks[i]);

        write(fd, buffer, strlen(buffer));
    }

    close(fd);

    printf("\nMarks written to file successfully.\n");

    fd = open("marks.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error opening file for reading");
        return 1;
    }

    char readBuffer[100];
    int bytesRead = read(fd, readBuffer, sizeof(readBuffer) - 1);

    if (bytesRead < 0) {
        perror("Error reading file");
        close(fd);
        return 1;
    }

    readBuffer[bytesRead] = '\0';

    char *token = strtok(readBuffer, "\n");
    while (token != NULL) {
        total += atoi(token);
        token = strtok(NULL, "\n");
    }

    printf("\nTotal Marks = %d\n", total);
    close(fd);
```